BUILD SPECIFICATION · DEVELOPER HANDOFF



Restaurant Billing SaaS

The Full Recipe

Everything we have to build: components, permissions, data model, rules, APIs, screens and the checklist

SERVICE WINDOW	PLATES	FINAL TASTING
13 Oct – 1 Nov 2026	Standard · Pro · Premium	AC-01 to AC-12

Gagan · Head Chef · Full Stack + DevOps Engineer
Aditya · Sous Chef · Frontend & UI Developer

BASED ON SRS V1.0 · PREPARED 12 OCTOBER 2026

THE FULL RECIPE

What We Are Building

Every component, rule, endpoint and screen due by 1 November 2026

01 What We Are Building

02 Components

03 Roles and Permissions

04 Plan Entitlements

05 Data Model

06 Status Lifecycles

07 Business Rules

08 Cloud API

09 Hub API and WebSocket Events

10 Offline and Sync

11 Super Admin Portal Screens

12 POS App Screens by Role

13 Printing and Documents

14 Non-Functional Requirements

15 Build Checklist

Based on SRS v1.0 and the Kitchen Playbook · Prepared 12 Oct 2026

COURSE ONE

01 What We Are Building

> We are building five deliverables that together make one multi-tenant restaurant billing product, sold in three plans. This spec lists everything that must exist by 1 Nov 2026; it follows SRS v1.0 plus the defaults agreed in the Kitchen Playbook.

#	Deliverable	Runs on	Used by
1	Cloud API + database	Linux server, India region	Every other component
2	Super Admin portal	Browser (Flutter Web)	Platform Super Admin, Platform Admins
3	POS app	Windows + Android (Flutter)	Owner, IT Admin, Cashier, Waiter, Chef, Auditor
4	Branch hub (inside the POS app)	Windows cashier PC, or the single Standard device	Every device in one branch
5	Installers, CI/CD, docs	GitHub Actions, server	Client ops team, end users

What each plan unlocks (each plan includes everything in the one before it):

- **Standard:** one restaurant, tables, dine-in and takeaway orders, menu with images and tax, bills, discounts, tips to waiter, reversals with reason, printed receipts, bill history, works offline on one device.
- **Professional:** multiple floors with their own table layouts, cross-floor transfers, several cashiers at once, waiter Android app, branch hub on the restaurant LAN with offline queue and cloud sync.
- **Premium:** franchise with multiple branches, consolidated franchise bill, IT Admin / Chef / Auditor roles, branch-scoped permissions, shifts and breaks, Kitchen Display System, chef recipes and menu approval, inventory with stock lifecycle, consumption reports.

Out of scope for 1 Nov (v1.1): payment-gateway auto-collection, split bill by items, FIFO valuation and batch/expiry, payroll, regional languages, hub failover, iOS.

COURSE TWO

02 Components

> The codebase is one monorepo with three apps and four shared packages; Gagan owns the server side, Aditya owns every screen.

Component	Path	What it does	Tech	Owner
Cloud API	`apps/api`	Tenants, auth, RBAC, MFA, licenses, audit, menu/config master data, sync ingest, reports, franchise invoices	NestJS, Prisma, PostgreSQL 16 with row-level security	Gagan (Aditya: menu/table CRUD)
Super Admin portal	`apps/admin_web`	Platform admin web console	Flutter Web	Aditya
POS app	`apps/pos_app`	All restaurant-side screens for every role; switches into Hub mode on the designated device	Flutter (Windows, Android), Riverpod, go_router	Aditya (UI), Gagan (hub wiring)
Branch hub server	`packages/hub_server`	Local HTTP + WebSocket server, SQLite store, single writer for live operations, outbox sync, printing	Dart `shelf`, `drift`	Gagan
Core	`packages/core`	Shared models, money in paise, GST and totals, validation, golden test vectors	Dart	Gagan

Component	Path	What it does	Tech	Owner
UI kit	`packages/ui_kit`	"House Plating" design system: tokens and components	Flutter	Aditya
Contracts	`packages/contracts`	OpenAPI spec generated from the API plus mock JSON fixtures	OpenAPI 3.1	Gagan
Infra	`infra/`	Docker Compose, Caddy, backup/restore scripts	Docker, Caddy	Gagan
CI/CD	`.github/workflows`	Lint, test, build API image, Windows installer, Android APK/AAB, web deploy, secret scan	GitHub Actions	Gagan
Object storage	external	Menu images (WebP, max 1 MB), generated PDFs	S3 Mumbai or Cloudflare R2	Gagan

COURSE THREE

03 Roles and Permissions

> Eight roles in two tiers: platform roles manage tenants, restaurant roles run a restaurant. Every check below is enforced by the API or hub; the UI only hides what the server already refuses.

Key: Y = allowed · R = read only · B = own branch(es) only · G = only if granted to that admin · P = needs owner/manager PIN · — = refused

Action	Super Admin	Platform Admin	Owner	IT Admin	Cashier	Waiter	Chef	Auditor
Create / suspend / reactivate restaurant	Y	G	—	—	—	—	—	—
Issue, renew, change, revoke license	Y	G	R	R	—	—	—	R
Invite platform admins, assign roles	Y	G	—	—	—	—	—	—
Reset another admin's MFA (step-up)	Y	G	—	—	—	—	—	—
View platform audit log	Y	G	—	—	—	—	—	—
Manage restaurant profile, tax settings	—	—	Y	Y	—	—	—	R
Manage staff users and roles	—	—	Y	Y	—	—	—	R
Pair devices, set hub, technical settings	—	—	Y	Y	—	—	—	—
Manage floors and tables	—	—	Y	Y	—	—	—	R
Submit menu item / ingredient change	—	—	Y	Y	Y	—	—	—
Approve recipe and ingredient changes	—	—	Y	—	—	—	Y	—
Publish menu changes and prices	—	—	Y	Y	—	—	—	—
Create / edit orders	—	—	Y	—	Y	B (assigned tables)	—	—
Transfer order to another table	—	—	Y	—	Y	B	—	—
Apply discount	—	—	Y	—	G (permission `discount.apply`)	—	—	—

Action	Super Admin	Platform Admin	Owner	IT Admin	Cashier	Waiter	Chef	Auditor
Reverse item not yet sent to kitchen	—	—	Y	—	Y	Y	—	—
Reverse item already sent to kitchen	—	—	Y	—	P	—	—	—
Take payment, close bill, assign tip	—	—	Y	—	Y	—	—	—
Adjust a paid bill (adjustment record)	—	—	Y	—	P	—	—	—
Kitchen queue: accept / preparing / ready	—	—	R	—	R	R	Y	—
Create and version recipes	—	—	R	—	—	—	Y	R
Inventory receipts, transfers, adjustments, wastage	—	—	Y	Y	—	—	Y	R
Shifts, breaks, attendance setup	—	—	Y	Y	—	—	—	R
Clock in / out, start / end break	—	—	Y	Y	Y	Y	Y	—
Reports and dashboards	—	—	Y (all branches)	B	own shift	own tips	kitchen only	R (authorised)
Restaurant audit log	—	—	Y	Y	—	—	—	R

Rules that apply to every role

- No one can grant a permission they do not hold, or change their own roles (self-escalation guard).
- Auditor is read-only everywhere: every write route returns 403 for the auditor role, tested across all routes (AC-11).
- In Premium, every restaurant role is scoped to one or more branches; queries filter by `branch_id`.
- Every permission change, MFA reset and status change writes an audit event with actor, reason and before/after values.

COURSE FOUR

04 Plan Entitlements

> Each feature has a flag (e.g. `kds`) carried inside the signed license token; the API guard `@RequiresFeature()` and the hub both refuse a flagged action when the tenant's plan lacks it. Limits are stored per plan as config, so the client can set the numbers.

Feature flag	What it gates	Standard	Professional	Premium
`tables`	Tables, free/occupied status, table transfer	Y	Y	Y
`orders.dinein`, `orders.takeaway`	Order types	Y	Y	Y
`menu`	Categories, items, images, tax per item	Y	Y	Y
`billing`	Bills, taxes, discounts, tips, reversals, adjustments, print	Y	Y	Y
`offline.single`	Device runs as its own hub	Y	Y	Y
`floors`	Multiple floors, per-floor layout, cross-floor transfer	—	Y	Y
`multi_cashier`	More than one concurrent cashier session	—	Y	Y
`waiter_app`	Waiter role on Android	—	Y	Y
`lan_hub`	Other devices pair to the hub over Wi-Fi	—	Y	Y

Feature flag	What it gates	Standard	Professional	Premium
`franchise`	Multiple branches, consolidated dashboard and invoice	—	—	Y
`roles.extended`	IT Admin, Chef, Auditor roles; branch-scoped RBAC	—	—	Y
`kds`	Kitchen Display System	—	—	Y
`recipes`, `menu.approval`	Recipe versions, chef approval workflow	—	—	Y
`inventory`	Departments, stock, receipts, transfers, wastage, consumption	—	—	Y
`shifts`	Shift templates, breaks, attendance	—	—	Y

Limit (configurable)	Default Standard	Default Professional	Default Premium
Branches	1	1	Set per franchise
Floors per branch	1	Set by client	Set by client
Paired devices per branch	1	Set by client	Set by client
Concurrent cashier sessions	1	Set by client	Set by client
Staff users per branch	Set by client	Set by client	Set by client

Expiry behaviour: for 7 days after license end date the tenant keeps full access with a renewal banner; after that it is read-only (history and reports visible, no new orders or bills). A blocked or suspended tenant is read-only immediately. Historical records are never deleted on suspension.

COURSE FIVE

05 Data Model

> 43 entities. All ids are UUIDv7; all money is integer paise; all times are UTC. Every restaurant-scoped row carries `tenant_id`, and branch-scoped rows also carry `branch_id`. **Writer** says which side creates and changes the row; the other side only reads a synced copy.

Platform and tenancy (cloud)

Entity	Key fields	Writer
PlatformAdmin	email, name, password_hash, status, last_login_at	Cloud
PlatformRole	name, permission keys	Cloud
Permission	key (e.g. `restaurant.suspend`), description, tier (platform / restaurant)	Cloud (seeded)
MFAEnrollment	owner (admin or user), totp_secret_encrypted, recovery_code_hashes[], enrolled_at, reset_by, reset_at	Cloud
RestaurantTenant	code, name, owner_email (verified), gstin, address, status (pending / active / suspended / blocked), status_reason	Cloud
Franchise	tenant_id, name, billing_contact	Cloud
Branch	tenant_id, franchise_id, code (e.g. BR01), name, address, gstin, settings JSON	Cloud
SubscriptionPlan	name, feature_flags[], limits JSON, price_paise, billing_cycle	Cloud
License	tenant_id, plan_id, starts_at, ends_at, grace_days, status, signed_token, issued_by, reason	Cloud
PaymentRecord	tenant_id, license_id, amount_paise, method, reference, paid_at, recorded_by	Cloud
FranchiseInvoice	franchise_id, period (month), lines (branch, plan, amount), total_paise, pdf_url	Cloud

People and devices (cloud)

Entity	Key fields	Writer
User	tenant_id, name, phone/email, pin_hash (for PIN approvals), password_hash, status	Cloud
UserRoleAssignment	user_id, role, branch_ids[], extra_permissions[]	Cloud
Device	branch_id, name, platform (windows / android), is_hub, device_token_hash, paired_at, last_seen_at	Cloud (pairing via hub)
StaffShift	branch_id, user_id, shift template, scheduled_start, scheduled_end, clock_in, clock_out	Hub
BreakRecord	shift_id, started_at, ended_at, overrun_flag	Hub

Menu, recipes, layout (cloud)

Entity	Key fields	Writer
Floor	branch_id, name, sort_order	Cloud
Table	floor_id, number/name, seats, layout_x, layout_y	Cloud (status lives on hub)
MenuCategory	branch_id or tenant-wide, name, sort_order	Cloud
MenuItem	category_id, name, description, unit, quantity, price_paise, tax_rule_id, available, image_id	Cloud
MenuImage	storage_key, width, height, bytes	Cloud
TaxRule	name, rate_bp (basis points, e.g. 500 = 5%), split (CGST/SGST)	Cloud
MenuChangeRequest	item / ingredient payload, submitted_by, status (submitted / chef_approved / rejected / published), reviewer, reason	Cloud
Recipe	menu_item_id, version, instructions, status, created_by	Cloud
RecipeIngredient	recipe_id, inventory_item_id, quantity, unit	Cloud

Live operations (hub, synced up)

Entity	Key fields	Writer
Order	branch_id, type (dine_in / takeaway), table_id, waiter_id, status, opened_by, opened_at, version	Hub
OrderLine	order_id, menu_item_id, name_snapshot, price_snapshot_paise, tax_rate_snapshot_bp, qty, notes, status (pending / sent / reversed), sent_at	Hub
OrderStatusHistory	order_id, from, to, actor, at	Hub
TableTransfer	order_id, from_table, to_table, actor, at	Hub
KitchenTicket	order_id, lines[], status, accepted_at, preparing_at, ready_at, served_at	Hub
Bill	order_id, invoice_no, subtotal_paise, discount_paise, cgst_paise, sgst_paise, tip_paise, round_off_paise, total_paise, status, closed_at	Hub
BillAdjustment	bill_id, type (reversal / correction), amount_paise, reason, approved_by, at	Hub
Payment	bill_id, mode (cash / upi / card), amount_paise, reference, idempotency_key (unique), at	Hub
TipAllocation	bill_id, waiter_id, amount_paise	Hub
InvoiceCounter	branch_id, financial_year, last_number	Hub

Inventory (hub, synced up)

Entity	Key fields	Writer
InventoryDepartment	branch_id, name (kitchen, bar, store)	Cloud
InventoryItem	name, base_unit (g / ml / pcs), purchase_unit, factor, low_stock_level	Cloud
StockMovement	item_id, department_id, type (receipt / transfer_in / transfer_out / consumption / adjustment / wastage), quantity, ref (receipt, ticket, recipe version), reason, actor, at	Hub
SupplierReceipt	department_id, supplier_name, lines (item, qty), received_by, at	Hub

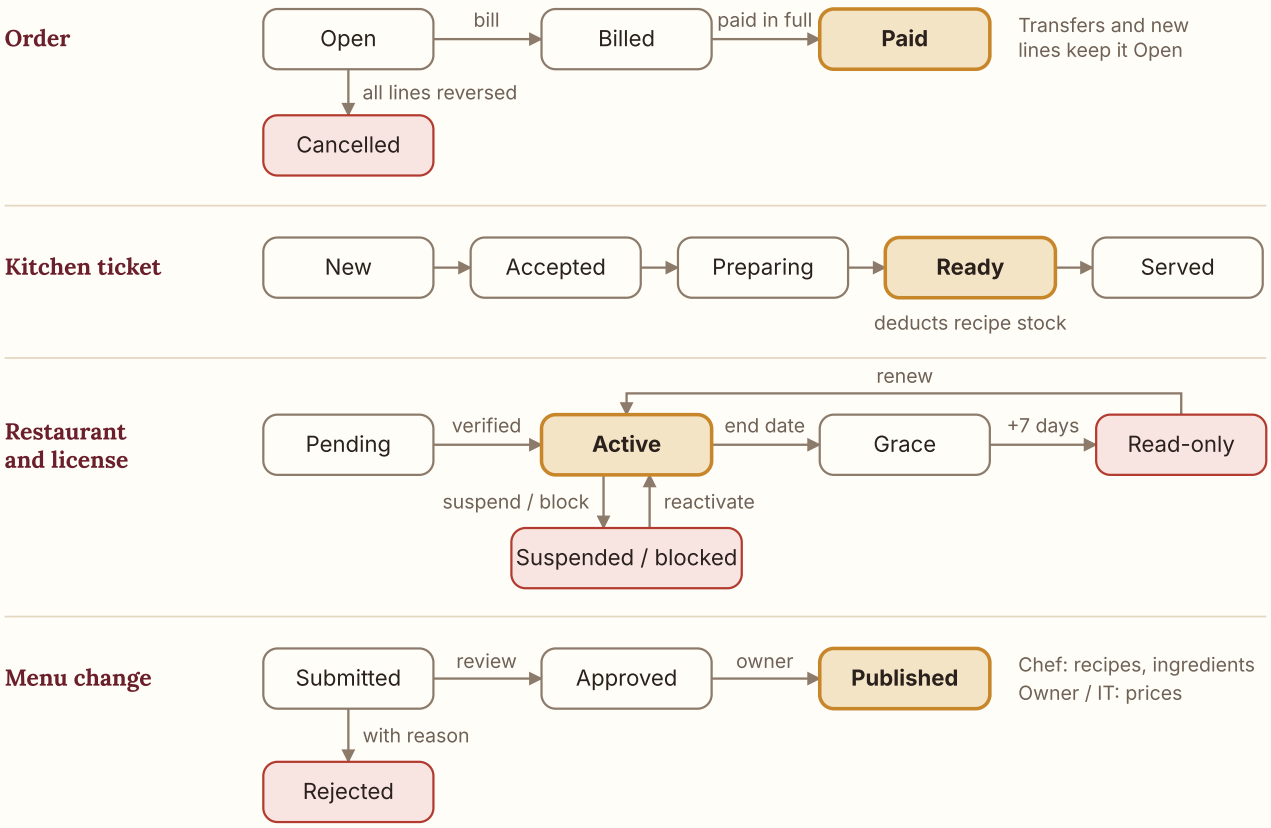
Audit and sync

Entity	Key fields	Writer
AuditEvent	actor_id, tenant_id, branch_id, action, object_type, object_id, before JSON, after JSON, reason, device_id, session_id, at	Both (append-only)
SyncOutbox	event_id, entity, entity_id, op, payload, created_at, pushed_at, attempts, last_error	Hub
SyncConflict	event_id, reason, payload, detected_at, resolved_by, resolution	Cloud
Notification	recipient, type (e.g. mfa_reset), payload, sent_at	Cloud

COURSE SIX

06 Status Lifecycles

> Four records have fixed states, and the server rejects any transition not drawn here; every transition writes a status-history row and an audit event.



Every record moves forward through a fixed set of states. Saffron states are the normal end points; red states block further work until someone acts.

Highlighted states are the normal end points; red states block further work until someone acts. A paid order can still receive a bill adjustment, which is a separate record, not a state change.

COURSE SEVEN

07 Business Rules

> These rules decide whether a bill is right. They live in `packages/core` (Dart) and are mirrored in the API only for reconciliation, both tested against the same golden vectors.

Money and tax

1. All amounts are integer paise. No floating point anywhere in money paths.
2. Line amount = price snapshot × quantity. The price and tax rate are copied onto the line when it is added, so later menu changes never alter an open or closed bill.
3. Discount (percentage or flat) applies to the taxable subtotal before tax, and is spread across lines in proportion to their amount; the last line absorbs the rounding remainder.

4. GST per line = discounted line amount × rate, rounded half-up to the paisa, then split equally into CGST and SGST (any odd paisa goes to CGST).
5. Tip is added after tax and is not taxed.
6. Round-off to the nearest rupee is a separate visible line on the bill (between −50 and +49 paise).
7. Grand total = subtotal − discount + CGST + SGST + tip + round-off.

Orders and tables

1. A table has at most one open order. Opening an order marks the table occupied; closing or cancelling the order frees it.
2. Table transfer moves the whole order in one transaction: it fails with a conflict if the target table is occupied, preserves every line and timestamp, writes a `TableTransfer` row and an audit event, and can never leave the order on two tables (AC-04).
3. Lines are sent to the kitchen explicitly (KOT). A sent line can no longer be edited, only reversed.
4. Every order row has a version number; a write with a stale version is rejected so two terminals cannot overwrite each other (AC-07).

Reversals and adjustments

1. Reversing a line requires a reason; a request without one is rejected (AC-05).
2. Unsent lines: cashier or waiter may reverse. Sent lines: needs an owner/manager PIN.
3. A reversal records actor, time, line, amount and reason, and recomputes totals.
4. A closed (paid) bill is never edited. Corrections create a `BillAdjustment` with reason and approver, and appear on reports.
5. A line reversed after the kitchen marked it Ready creates a wastage stock movement.

Payments and tips

1. One bill may be paid across several modes (cash, UPI, card) until the paid amount equals the total.
2. Every payment request carries an idempotency key from the terminal; repeating a key returns the first result, never a second payment (AC-07).
3. Cash payments record amount tendered and change due.
4. A tip is assigned to exactly one waiter selected at payment; for dine-in it defaults to the order's waiter (AC-06).

Invoice numbers

1. Format `<branch code>/<FY>/<6-digit sequence>`, e.g. `BR01/26-27/000123`. Financial year runs April to March.
2. Numbers are issued by the branch hub only, at bill close, gap-free and never reused.

Discounts

1. Needs permission `discount.apply`.
2. Above a configurable limit (default 10%), a reason is mandatory.

Kitchen and inventory

1. KOT creates a `KitchenTicket`. Statuses move forward only: new → accepted → preparing → ready → served.
2. When a ticket is marked Ready, each line's published recipe deducts its ingredients as `consumption` stock movements (AC-10).
3. Stock is tracked per department in the item's base unit; purchases convert using the item's purchase-unit factor.
4. Low-stock alert fires when on-hand quantity drops to or below the item's low-stock level.

Menu approval

1. Cashier or IT Admin submits an item or ingredient change. The chef approves or rejects recipe and ingredient changes. Price changes go to Owner or IT Admin. The Owner publishes.

2. Publishing bumps the menu version; hubs pull it on their next sync.

Licensing

1. Grace: 7 days of full access after license end, then read-only.
2. Blocked or suspended: read-only immediately; reason and actor are recorded.
3. The hub refuses gated actions if the signed token's signature fails or its expiry plus grace has passed (AC-02).

08 COURSE EIGHT Cloud API

> REST over HTTPS at `/api/v1`, JSON bodies, JWT bearer auth, OpenAPI generated into `packages/contracts`. Every route declares its permission and, where relevant, its feature flag; every write route records an audit event.

Auth and MFA

Method	Path	Purpose
POST	`/auth/login`	Email/password; returns `mfa_required` + challenge when MFA is on
POST	`/auth/mfa/verify`	TOTP or recovery code; returns access + refresh tokens
POST	`/auth/refresh`	Rotate refresh token
POST	`/auth/logout`	Revoke refresh token
POST	`/auth/step-up`	Re-verify password + TOTP; returns short-lived step-up token
POST	`/me/mfa/enroll` · `/me/mfa/confirm`	Start TOTP enrolment, confirm with first code, issue 10 recovery codes
POST	`/auth/pin/verify`	Owner/manager PIN approval for privileged hub actions

Platform administration

Method	Path	Purpose
GET / POST	`/platform/admins`	List / invite platform admins
PATCH	`/platform/admins/:id/roles`	Assign roles (self-escalation guard)
POST	`/platform/admins/:id/mfa/reset`	Reset MFA; needs step-up token; audited + notification (AC-03)
GET	`/platform/audit`	Platform audit log with filters
GET / POST	`/restaurants`	List with search/filter (status, plan, expiry, owner) / register
POST	`/restaurants/verify-email`	Confirm owner email; creates tenant + license (AC-01)
GET / PATCH	`/restaurants/:id`	Detail / edit
POST	`/restaurants/:id/status`	Activate, suspend, block, reactivate with reason
GET / POST	`/plans`	Plans, feature flags, limits
POST	`/restaurants/:id/licenses`	Issue, renew, upgrade, downgrade, revoke (action + reason)
GET	`/restaurants/:id/licenses/current/token`	Current signed license token
POST	`/restaurants/:id/payments`	Record a subscription payment
GET	`/franchises/:id/invoices?month=`	Consolidated franchise invoice (AC-12)

Restaurant setup

Method	Path	Purpose
GET / PATCH	`/tenant/profile`	Restaurant profile, GSTIN, tax settings
CRUD	`/branches`	Branches (Premium)
CRUD	`/users` · `/users/:id/roles`	Staff and branch-scoped roles
CRUD	`/floors` · `/tables`	Floors and tables with layout positions
CRUD	`/menu/categories` · `/menu/items`	Menu master data
POST	`/menu/images`	Signed upload URL; server converts to WebP, max 1 MB
CRUD	`/tax-rules`	GST rates
POST / GET	`/menu/change-requests`	Submit / list menu change requests
POST	`/menu/change-requests/:id/review`	Chef approve / reject
POST	`/menu/publish`	Owner publishes; bumps menu version
CRUD	`/recipes` · `/recipes/:id/versions`	Recipes with ingredient lines
CRUD	`/inventory/departments` · `/inventory/items`	Inventory master data
CRUD	`/shift-templates`	Shift and break templates
GET	`/audit`	Restaurant audit log

Devices and sync

Method	Path	Purpose
POST	`/devices/register-hub`	Hub device registers with branch credentials
GET	`/sync/pull?since=<cursor>`	Config changes since cursor: menu, tax, users, roles, tables, recipes, license token
POST	`/sync/push`	Batch of hub events; dedupe by event id; returns accepted / duplicate / rejected per event
GET	`/sync/status`	Last push time, pending count, conflicts per branch
GET	`/sync/conflicts`	Rejected events for review

Reports (from synced data)

Method	Path	Purpose
GET	`/reports/sales?from=&to=&branch=`	Daily sales, payments by mode, taxes
GET	`/reports/dishes`	Quantity and revenue per dish
GET	`/reports/tips`	Tips by waiter
GET	`/reports/inventory`	Consumption, movements, wastage, low stock
GET	`/reports/reconciliation`	Hub totals vs cloud recompute, mismatches
GET	`*.csv` variants	CSV export of each report

COURSE NINE

09 Hub API and WebSocket Events

> The hub serves the branch over Wi-Fi at `https://<hub-ip>:8443/hub/v1` with a self-signed certificate that clients pin at pairing. Devices authenticate with a device token plus the signed-in user's session; every

> write checks the user's permission and the license flags.

Hub endpoints

Method	Path	Purpose
GET	`/pairing/info`	Hub name, branch, cert fingerprint (shown as QR on the hub screen)
POST	`/pairing/claim`	Device sends one-time code; receives device token
POST	`/session/login`	Staff login by user + PIN or password on a paired device
GET	`/state`	Snapshot: floors, tables with status, open orders, KDS queue
POST	`/orders`	Open order (dine-in with table, or takeaway)
POST	`/orders/:id/lines`	Add lines (price + tax snapshot)
PATCH	`/orders/:id/lines/:lineId`	Change quantity or notes on an unsent line
POST	`/orders/:id/kot`	Send pending lines to kitchen; creates ticket
POST	`/orders/:id/lines/:lineId/reverse`	Reverse with reason; PIN if already sent
POST	`/orders/:id/transfer`	Move to another table (conflict check)
POST	`/orders/:id/bill`	Generate bill: totals, discount, invoice number
POST	`/bills/:id/payments`	Add payment; header `Idempotency-Key` required
POST	`/bills/:id/tip`	Assign tip to a waiter
POST	`/bills/:id/adjustments`	Post-payment correction with reason + PIN
POST	`/bills/:id/print`	Print receipt; returns PDF if no printer
GET	`/bills?date=`	Bill history for the day
POST	`/kds/tickets/:id/status`	Accept, preparing, ready, served
POST	`/inventory/receipts` · `/transfers` · `/adjustments` · `/wastage`	Stock movements
POST	`/shifts/clock-in` · `/clock-out` · `/break/start` · `/break/end`	Attendance
GET	`/sync/status`	Pending, pushed, failed counts; last pull time
GET	`/health`	Liveness for clients and the status banner

WebSocket events (server to clients)

Clients connect to `wss://<hub-ip>:8443/hub/v1/events`, receive a full `state.snapshot` on connect, then deltas. Each event carries `event_id`, `branch_id`, `at` and the changed object.

Event	Sent when	Who listens
`state.snapshot`	Client connects or reconnects	All
`table.status_changed`	Order opened, closed or transferred	Cashier, waiter, owner
`order.updated`	Lines added, changed, reversed; KOT sent	Cashier, waiter
`order.transferred`	Table transfer succeeds	Cashier, waiter
`bill.created` · `bill.paid` · `bill.adjusted`	Bill lifecycle	Cashier, owner
`kds.ticket_created`	KOT sent	Chef / KDS

Event	Sent when	Who listens
`kds.ticket_status`	Accept, preparing, ready, served	KDS, waiter, cashier
`inventory.low_stock`	Item at or below low-stock level	Chef, owner, IT admin
`menu.version_changed`	Hub pulled a new menu version	All POS screens
`license.state`	Token refreshed, grace started, read-only	All
`sync.status`	Pending / failed counts change	Cashier, owner, IT admin

COURSE TEN

10 Offline and Sync

> The restaurant keeps working with no internet as long as the hub PC and Wi-Fi are up; the cloud catches up when the link returns, and nothing is lost or counted twice.

Who writes what

Data	Single writer	Direction
Tenants, licenses, users, roles, plans, menu, tax, tables, recipes	Cloud	Cloud → hub by `/sync/pull` cursor
Orders, bills, payments, reversals, transfers, tickets, stock movements, shifts	Branch hub	Hub → cloud by `/sync/push` outbox
Audit events	Side where the action happened	Hub events pushed up; cloud stores permanently

Pairing a device

1. IT Admin sets one Windows PC as hub; the app starts the hub server and auto-starts with Windows.
2. Hub screen shows a QR code: hub IP, port, one-time code (valid 10 minutes), certificate fingerprint.
3. New device scans it (or types IP + code), checks the fingerprint, and receives a device token bound to the branch.
4. Hub found later by mDNS name; falls back to the saved IP.

Push (hub → cloud)

1. Every hub write and its outbox event are saved in the same SQLite transaction.
2. A worker sends batches of up to 200 events every 5 seconds while online, with exponential backoff up to 5 minutes while offline.
3. Cloud inserts each event once (unique `event_id`); replays return `duplicate`, never a second row (AC-08).
4. Events the cloud cannot accept (e.g. item deleted in cloud) are stored in `SyncConflict` and shown on the reconciliation report; the hub marks them failed and stops retrying them.

Pull (cloud → hub)

1. Hub pulls on start, every 60 seconds while online, and after any push.
2. The response includes the current signed license token; the hub verifies it before saving.
3. A new menu version applies to new lines only; lines already on orders keep their snapshots.

Failure cases the build must handle

Case	Expected behaviour
Internet down, Wi-Fi up	Everything works on the LAN; banner shows "Offline, N changes waiting"; sync resumes automatically

Case	Expected behaviour
Hub PC restarts	SQLite state survives; clients reconnect and receive `state.snapshot`; no open order lost
A client device restarts or drops Wi-Fi	Client retries with the same idempotency keys; hub returns earlier results; no duplicates
Hub PC switched off	Clients show "Kitchen hub offline" and stop taking orders (no second writer); recommend UPS
Wi-Fi itself fails	No device-to-device communication; the hub PC can still bill at its own screen
License expires while offline	Hub enforces expiry + grace from the signed token without needing the cloud
Same batch pushed twice	Cloud stores it once; hub marks events pushed

COURSE ELEVEN

11 Super Admin Portal Screens

> 11 screens in Flutter Web, desktop-first layout. Every screen has loading, empty and error states; every destructive action opens a reason dialog.

#	Screen	Shows	Actions
P1	Login	Email, password	Sign in; forgot password
P2	MFA	6-digit code field; "use recovery code" link	Verify; first-time enrolment shows QR + 10 recovery codes to save
P3	Dashboard	Counts: active, suspended, expiring in 30 days, in grace; recent audit events	Jump to filtered restaurant list
P4	Restaurants list	Table: name, code, owner, plan, status chip, license end, branches	Search; filter by status, plan, expiry, owner; sort; paginate
P5	Register restaurant	Owner email, name, GSTIN, address, plan	Send verification email; shows pending until verified
P6	Restaurant detail	Profile, current license, branches, devices, payment history, audit tab	Edit profile; activate / suspend / block / reactivate with reason
P7	License actions	Current plan, dates, grace, features	Issue, renew, upgrade, downgrade, revoke with reason; record payment
P8	Plans	Plans with feature flags, limits, price	Create and edit plans
P9	Platform admins	Admin list with roles and MFA status	Invite admin; assign roles; deactivate
P10	MFA reset	Target admin, warning text	Step-up prompt (password + own TOTP), reason, confirm; success toast notes the notification sent
P11	Audit viewer	Time, actor, action, object, reason; before/after diff on expand	Filter by actor, action, restaurant, date; export CSV

COURSE TWELVE

12 POS App Screens by Role

> 35 screens in one Flutter app on Windows and Android; after login, the user's role and the license flags decide which screens appear. Cashier screens target a 1366 × 768 landscape PC and 10-inch tablets; waiter screens target a phone held in one hand.

Shared

#	Screen	Shows	Actions	Plan
S1	Device setup	"This device is the hub" or "Pair with hub"	Enable hub mode; scan QR / enter IP + code	All
S2	Hub status (hub device)	Pairing QR, paired devices, sync counts, license state	Revoke a device; force sync	All
S3	Staff login	User tiles or ID; PIN pad	Log in; switch user	All
S4	Status bar (every screen)	Hub online/offline, sync pending/failed, license banner	Tap for details	All

Cashier

#	Screen	Shows	Actions	Plan
C1	Table map	Floors as tabs; tables coloured free / occupied / bill printed; running total on occupied	Open order; open takeaway; switch floor	All (floors: Pro+)
C2	Order screen	Category rail, item grid with search, order panel with lines, qty steppers, notes	Add / change / remove unsent lines; send KOT; assign waiter	All
C3	Table transfer	Free tables across floors	Pick target; confirm; conflict message if taken	All
C4	Reversal dialog	Line, amount, reason field (required), PIN pad if sent	Confirm reversal	All
C5	Bill preview	Lines, subtotal, discount, CGST, SGST, tip, round-off, total	Apply discount (permission); generate bill	All
C6	Payment	Remaining amount; cash keypad with change due; UPI; card reference; split across modes	Add payment; finish	All
C7	Tip	Waiter picker (defaults to order's waiter), amount	Assign tip	All
C8	Bill history	Today's bills, search by number or table	Reprint; open adjustment (PIN)	All
C9	Adjustment dialog	Original bill, type, amount, reason	Post adjustment with PIN	All
C10	Shift summary	My bills, totals by payment mode, cash expected	Close shift	All

Waiter (Android phone)

#	Screen	Shows	Actions	Plan
W1	My tables	Assigned tables with status and ready-dish alerts	Open table	Pro+
W2	Take order	Category list, items with big + buttons, notes	Add lines; send KOT	Pro+
W3	Order status	Lines with kitchen status chips	Reverse unsent line; mark served	Pro+
W4	My tips	Today's tips	View	Pro+

Chef / Kitchen Display

#	Screen	Shows	Actions	Plan
K1	KDS queue	Ticket cards: table, items, qty, notes, timer, colour by age	Accept, preparing, ready, served	Premium
K2	Recipe editor	Item, version history, ingredient lines (item, qty, unit), steps	Save new version	Premium

#	Screen	Shows	Actions	Plan
K3	Approval inbox	Pending change requests with diff	Approve / reject with reason	Premium
K4	Kitchen stock	On-hand for kitchen department, low-stock list	Record wastage	Premium

Owner

#	Screen	Shows	Actions	Plan
O1	Dashboard	Today's sales, bills, average bill, top dishes, payment split; branch switcher in Premium	Drill into reports	All (branches: Premium)
O2	Reports	Sales, dishes, tips, taxes, inventory, reconciliation; date and branch filters	Export CSV / PDF	All (inventory: Premium)
O3	Menu management	Categories, items, prices, tax, images, availability	Edit; publish	All
O4	Menu publish	Approved changes waiting	Publish	Premium
O5	Staff	Users, roles, branches, PIN reset	Add / edit / deactivate	All
O6	Franchise invoice	Monthly invoice with branch lines	Download PDF	Premium

IT Admin

#	Screen	Shows	Actions	Plan
T1	Devices	Paired devices, last seen, platform, hub flag	Revoke; set hub	All
T2	Floors and tables editor	Drag tables on a grid; seats; numbers	Add / move / delete tables and floors	All (floors: Pro+)
T3	Printers	Printer IP, port, paper width, test print	Save; test	All
T4	Inventory	Departments, items, receipts, transfers, adjustments	Record movements	Premium
T5	Shifts setup	Shift templates, break windows, roster	Create; assign	Premium

Auditor

#	Screen	Shows	Actions	Plan
A1	Audit log	Restaurant audit events with filters and before/after	View; export	Premium
A2	Read-only reports	Same as O2, no edit controls	View; export	Premium

Every screen also needs: loading, empty and error states; keyboard shortcuts on Windows for the cashier (numbers, Enter to pay, F-keys for categories); touch targets of at least 48 dp on Android.

COURSE THIRTEEN

13 Printing and Documents

> Four printed outputs. Thermal prints go as ESC/POS bytes over TCP port 9100 to an 80 mm printer (48 characters per line); every print also has a PDF fallback built with the Dart `pdf` package.

Document	Printed when	Must contain
Kitchen order ticket (KOT)	Lines sent to kitchen	Table or takeaway no., waiter, time, items with qty and notes, "REPRINT" marker on reprints; no prices
Customer bill / tax invoice	Bill generated or reprinted	Restaurant name, address, GSTIN; invoice no. and date; table, waiter; lines (item, qty, rate, amount); subtotal, discount, CGST and SGST with rates, tip, round-off, total; payment modes; "Duplicate" on reprints
Shift / day summary	Cashier closes shift	Bills count, totals by payment mode, taxes, reversals and adjustments with reasons, expected cash
Franchise subscription invoice (PDF only)	Monthly, from cloud	Franchise details, period, one line per branch (plan, days, amount), consolidated total (AC-12)

Printer settings (IP, port, paper width, which printer gets KOT vs bill) are set per branch on the IT Admin printer screen and stored on the hub. A test-print button prints a sample of each layout.

COURSE FOURTEEN

14 Non-Functional Requirements

> Targets marked "proposed" are our defaults until the client confirms them (SRS §9 leaves them open).

Area	Requirement
Tenant isolation	Postgres row-level security on `tenant_id`; automated test that tenant A gets zero rows of tenant B on every endpoint
Authentication	Argon2id hashes; JWT access 15 min; rotating refresh tokens stored hashed; TOTP MFA mandatory for platform admins; 5 failed logins → 15-minute lockout
Secrets	Only in environment variables, validated at startup; `.env.example` committed, real values never; secret scan in CI
Transport	TLS on all cloud traffic (Caddy auto-certificates); LAN traffic TLS with certificate pinned at pairing
Rate limiting	Auth routes 10 per minute per IP; other API routes 300 per minute per user
Logging	Structured JSON; no passwords, tokens, MFA secrets or card data in logs or audit
Audit retention	3 years online, then archive (proposed)
Performance (proposed)	Hub writes under 150 ms on the LAN; change visible on other terminals within 1 s; cloud API p95 under 400 ms; POS cold start under 4 s
Capacity (proposed)	500 tenants, 20 devices per branch, 300 orders per branch per day on the first server
Backups (proposed)	Daily database snapshot kept 30 days + point-in-time recovery 7 days; RPO 15 min, RTO 4 h; restore drill before release
Monitoring	Sentry for API and Flutter crashes; Uptime Kuma on API health; alert on sync conflict spikes
Data residency	India region (AWS ap-south-1 or DigitalOcean Bangalore)
Devices	Windows 10/11 64-bit; Android 9+ phones and 8–10 inch tablets; Chrome or Edge for the portal
Images	Max 1 MB after upload, converted to WebP
Language	English in v1
Automated tests	Entitlements, RBAC and auditor read-only, money golden vectors, tenant isolation, offline disconnect/reconnect, duplicate push, duplicate payment, one happy + one failure test per acceptance criterion
Delivery	Signed Windows installer, Android AAB/APK, portal URL, deployment and restore guides, user guides per role

COURSE FIFTEEN

15 Build Checklist

> Tick each item when it is merged, on staging, and its tests pass. (G) = Gagan, (A) = Aditya.

Phase 1: Foundation (Days 1–5)

- [ ] Monorepo, workspaces, lint, `.env.example` , dev-secret script (G)
- [ ] Docker Compose with Postgres 16; staging server with Caddy (G)
- [ ] CI: API, Dart, Flutter, secret scan (G)
- [ ] Prisma schema, RLS migration, seed with 3 plans + Super Admin (G)
- [ ] Architecture doc, permission matrix, OpenAPI v1 (G)
- [ ] Design tokens, ui_kit components, screen map (A)
- [ ] Auth: login, refresh, logout, lockout, rate limits (G)
- [ ] MFA: enrol, verify, recovery codes, step-up, reset (G)
- [ ] Audit service, append-only DB grants (G)
- [ ] Restaurant register, verify email, lifecycle with reason (G)
- [ ] Plans, licenses, signed token, `@RequiresFeature()` , grace / read-only (G)
- [ ] Portal P1–P11 (A)
- [ ] Hub server skeleton, pairing, WebSocket, offline license check (G)
- [ ] POS shell: device setup, hub status, staff login, status bar S1–S4 (A)
- [ ] AC-01, AC-02, AC-03 tests green and demoed

Phase 2: Standard (Days 6–10)

- [ ] Menu, categories, tax rules, image upload; table CRUD (A, reviewed by G)
- [ ] Config pull sync cloud → hub (G)
- [ ] `core` money package + 30+ golden vectors, mirrored in API (G)
- [ ] Order engine: open, lines with snapshots, KOT, versioning (G)
- [ ] Bills, discounts, GST split, round-off, invoice numbers (G)
- [ ] Payments with idempotency, multi-mode, tips, reversals, adjustments (G)
- [ ] Table transfer in one transaction (G)
- [ ] ESC/POS + PDF printing; KOT and bill layouts (G + A)
- [ ] Outbox push, cloud dedupe, reconciliation report (G)
- [ ] Cashier screens C1–C10; owner O1, O3, O5 (A)
- [ ] AC-04, AC-05, AC-06 tests green and demoed

Phase 3: Professional (Days 11–13)

- [ ] Floors, per-floor layout, cross-floor transfer (G + A)
- [ ] Multi-cashier concurrency, duplicate-payment tests (G)
- [ ] Hub restart recovery, client retry queue, reconnect snapshot (G)
- [ ] Offline harness: WAN cut, LAN cut, duplicate and conflict simulation (G)
- [ ] Waiter screens W1–W4; IT Admin T1–T3 (A)
- [ ] AC-07, AC-08 tests green and demoed

Phase 4: Premium (Days 14–17)

- [ ] Franchise and branches, branch-scoped RBAC, auditor read-only (G)
- [ ] KDS tickets and status events (G); KDS screen K1 (A)
- [ ] Recipes with versions; menu change requests and approval (G); K2, K3, O4 (A)
- [ ] Inventory master data, movements, recipe deduction at Ready, low stock (G); K4, T4 (A)

- [ ] Shifts, breaks, attendance (G); T5 (A)
- [ ] Reports and CSV export (G); O2, A1, A2 (A)
- [ ] Consolidated franchise invoice (G); O6 + PDF layout (A)
- [ ] AC-09, AC-10, AC-11, AC-12 tests green and demoed

Phase 5: Hardening and handoff (Days 18–20)

- [ ] Security pass: RLS tests, rate limits, dependency audit, secret scan of history (G)
- [ ] k6 load test against capacity targets (G)
- [ ] Backup + restore drill logged (G)
- [ ] Production deploy, monitoring and alerts (G)
- [ ] Signed Windows installer, Android AAB/APK (G)
- [ ] UI polish, tablet layouts, accessibility (A)
- [ ] User guides per role with screenshots (A); admin ops guide (G)
- [ ] Client UAT; full AC-01 to AC-12 demo; sign-off